Lab 10: Data Integrity with MySQL Constraints

In this lab exercise/tutorial, you will recreate the `university` database with the appropriate constraints.

Start by logging in to your MysQL server, creating a new database called `university_dup`. All of the following operations are to be performed on the new database.

Each task ends with a test component. Write the query/command (as applicable) and the output of each test, and explain the results as required.

What to submit?

Submit a PDF file, #10_lastname.pdf, containing your answers to the questions given in each task in the submission bin for this activity in the LMS. Like in previous labs, make sure that SQL code is properly formatted; you may use CodeBlocks GDocs extension. Use this template: Answer Sheet Template - Lecture.

Task 1: Primary Keys (Identifying Records)

Recall that a **Primary Key** is the unique identifier for a table. It must be unique and cannot contain `NULL` values. In the University schema, every department must have a unique name to avoid confusion. Using a **Table-Level** constraint allows you to name the key, making it easier to reference or drop later.

Example:

```sql
CREATE TABLE department (
    dept_name VARCHAR(20),
    building VARCHAR(15),
    budget NUMERIC(12, 2),
```

```
    CONSTRAINT pk_dept PRIMARY KEY (dept_name)
);
```

Test:

Run these two queries. What is the result? Explain why this happened.

```SQL
INSERT INTO department VALUES ('Comp. Sci.', 'Taylor', 100000);
INSERT INTO department VALUES ('Comp. Sci.', 'Watson', 80000);
```

Task 2: Not Null & Unique (Data Quality)

Discuss: While a Primary Key identifies the row, other columns often require data to exist (**NOT NULL**) or stay unique (**UNIQUE**) without being the main identifier. For example, every instructor must have a name, and their internal `ID` must be unique.

Example:

```SQL
CREATE TABLE instructor (
    ID VARCHAR(5) NOT NULL,
    name VARCHAR(20) NOT NULL,
    dept_name VARCHAR(20),
    salary NUMERIC(8, 2),
    UNIQUE (ID)
);
```

Test:

Try to insert an instructor without a name. MySQL should block this. What is the result? Explain why this happened.

```SQL
INSERT INTO instructor (ID, dept_name, salary) VALUES ('10101',
'Biology', 65000);
```

Task 3: Check Constraints (Domain Integrity)

Discuss: A **CHECK** constraint defines a logical filter for your data. In MySQL, any expression that returns `TRUE`, `FALSE`, or `UNKNOWN` can be used. Common conditions include:

1. **Comparison**: `salary > 0`

2. **Range**: `year BETWEEN 1701 AND 2100`
3. **List Validation**: `semester IN ('Fall', 'Winter', 'Spring', 'Summer')`
    a. other types like int are supported in list as well.
4. **Multi-column Logic**: `CHECK (end_time > start_time)`

Note: Requires MySQL 8.0.16 or higher to be enforced.

Example:

Add a rule to the `department` table ensuring the budget is positive and the building name isn't an empty string.

```SQL
ALTER TABLE department
ADD CONSTRAINT chk_dept_rules
    CHECK (budget > 0 AND building <> '');
```

Test:

Try to create a "Ghost Department" with a negative budget and an empty building. What is the result? Explain why this happened.

```SQL
INSERT INTO department (dept_name, building, budget)
VALUES ('Alchemy', '', -500);
```

Task 4: Default Values (Automation)

Discuss: The **DEFAULT** constraint provides a fallback value when no data is provided during an `INSERT`. This is useful for fields like `credits`, where most courses follow a standard.

First, we create the `course` table with a basic structure.

Create the Course Table

```SQL
CREATE TABLE course (
    course_id VARCHAR(8) PRIMARY KEY,
    title VARCHAR(50) NOT NULL,
    dept_name VARCHAR(20),
```

```
    credits INT
);
```

Example:

```SQL
ALTER TABLE course
ALTER credits SET DEFAULT 3;
```

Test:

Insert a course without specifying credits and verify that "3" was assigned. What is the output of the select query?

```SQL
INSERT INTO course (course_id, title, dept_name)
VALUES ('CS-101', 'Intro to SQL', 'Comp. Sci.');

SELECT * FROM course WHERE course_id = 'CS-101';
```

Task 4.1: The Constraint Conflict (NOT NULL vs. DEFAULT NULL)

Discuss:

What happens when two rules disagree?

Create a table with an arbitrary column defined as `NOT NULL` (forbidding empty values) but set its `DEFAULT` to `NULL`.

Did MySQL allow you to do this? What is the output of MySQL? Show the create table query, and the output of MySQL.

Task 5: Referential Integrity (Foreign Keys)

Using the `course` table previously created, perform the following tests below. Later, we use `ALTER TABLE` to add a Foreign Key with specific rules.

Test 5.1:

1. Insert new department records, at least 2, in the department table.
2. Insert a new course with a dept_name that exists in the department table.
3. Then modify the department record in the department table corresponding to the course added in #2.
4. What happens? Is this ok? Explain your answer.

Test 5.2

1. Insert a new course with a department name that does not exist in the department table.
2. What happens? Is this ok? Explain your answer.

Defining a foreign key constraint allows us to handle both issues in Test 5.1 and 5.2. Recall that foreign keys link tables. **Referential actions** define how "child" records react when "parent" records change:

- **CASCADE**: The change flows down (Update parent, Update child).
- **SET NULL**: The link is broken, but the child record remains with a NULL value.
- **RESTRICT**: (Default) Prevents the change if child records exist.

Example:

Link `course` to `department`. If a department name changes, update courses; if a department is deleted, set the course's department to `NULL`.

```sql
ALTER TABLE course
ADD CONSTRAINT fk_course_dept
FOREIGN KEY (dept_name) REFERENCES department(dept_name)
ON UPDATE CASCADE
ON DELETE SET NULL;
```

Test 5.3:

What is the output of each test? Explain why this happened.

1. **Update**: Change 'Comp. Sci.' to 'Computer Science' in `department`.
2. **Delete**: Delete 'Biology' from `department`.

Task 6: Managing and Dropping Constraints

To drop a constraint, you must first know its **Internal Name**. MySQL allows you to name constraints (e.g., `chk_salary`), but if you didn't provide one during `CREATE TABLE`, MySQL assigns a system-generated name (like `instructor_chk_1`).

The syntax for dropping depends on the *type* of constraint:

- **Primary Key:** `DROP PRIMARY KEY` (No name needed since there is only one).
- **Foreign Key:** `DROP FOREIGN KEY [name]`.
- **Check Constraint:** `DROP CHECK [name]`.
- **Unique/Index:** `DROP INDEX [name]`.

Example:

Suppose the University decides to remove the minimum salary restriction to allow for part-time teaching assistants.

Step A: Identify the Constraint Name

Run this command to see the "Raw SQL" used to build the table. Look for the `CONSTRAINT` lines.

```sql
SHOW CREATE TABLE department;
```

Step B: Drop the Specific Constraint

Use the name found in Step A (we'll assume it was named `chk_dept_rules`).

```sql
ALTER TABLE department
```

```
DROP CHECK chk_dept_rules;
```

Step C: Drop a Primary Key (Special Case)

If you need to restructure a table's ID system:

```
SQL
ALTER TABLE department

DROP PRIMARY KEY;
```

Test:

Verify the constraint is gone by performing an action that was previously blocked.

1. **Before Drop**: Attempting to set a department's building to an empty string `''` results in an error.
2. **After Drop**: Run the update again, for example, assuming dept_name = 'Comp. Sci.' exists.

```
SQL
UPDATE department SET building = '' WHERE dept_name = 'Comp. Sci.';
```

What happens after doing the update? What is the output? Explain why this happened.

Task 7: Final Challenge: The `favorite_courses` Table

Scenario:

The University wants to track which courses students enjoyed the most. Each

student can nominate several "favorite" courses, but they must have completed the course with a specific grade to qualify.

Create the `student` table:

```
SQL

create table student
  (ID           varchar(5),
   name         varchar(20) not null,
   dept_name    varchar(20),
   tot_cred     numeric(3,0) check (tot_cred >= 0),
   primary key (ID),
   foreign key (dept_name) references department (dept_name)
       on delete set null
  );
```

Task Requirements:

1. **Identity**: A student can have multiple favorites, but they cannot list the same course twice.
2. **Links**: Ensure that if a student is deleted from the `student` table, their favorites are also removed (**CASCADE**).
3. **The "Grade Gap" Constraint**: The university only accepts specific numeric grades for this list. The grade must be between **1.0 and 3.0**, and it must be in increments of **0.25** (e.g., 1.0, 1.25, 1.5... up to 3.0).
4. **Data Integrity**: The `course_id` must exist in the `course` table.

For each task, write a test query showing that the constraints are enforced. In your answer, add the output of the `show create` table command in your created table, the test queries used, and the output of each test query.

Final Note: Beyond Constraints – The Role of Assertions

While we have covered **Primary Keys**, **Foreign Keys**, and **CHECK constraints**, these are all "local" rules. They can only validate data within a single row or by looking at a direct parent-child relationship.

In a complex University system, you often encounter **Global Business Rules** (known in the SQL standard as **Assertions**) that span multiple tables. For example:

- *"A student cannot have more than 18 credits per semester."*
- *"A classroom cannot be booked if its capacity is less than the number of enrolled students."*
- *"A course cannot be added to `favorite_courses` unless the student has a passing record in the `takes` table."*

The MySQL Reality

MySQL **does not support standard SQL Assertions**. This means a `CHECK` constraint cannot run a `SELECT` query against another table to verify a condition.

Moving to Triggers

To enforce these "multi-table" rules, we must move from **Declarative Constraints** (rules we define in the table structure) to **Procedural Logic** using **Triggers**.

A **Trigger** is a named database object that is associated with a table and activates ("fires") when a specific event occurs (INSERT, UPDATE, or DELETE). In our next activity, we will learn how to:

1. **Intercept** an insert before it happens.
2. **Query** other tables (like `takes` or `section`) to validate the new data.
3. **Signal** a custom error to block the transaction if the business rule is violated.

Think of it this way: Constraints are the "automatic locks" on your doors, but Triggers are the "security guards" who check your ID and paperwork before letting you in.

Lab Complete. You now have a database that polices itself against duplicate IDs, orphaned records, and invalid grades!